

操作系统原理

第六章：CPU调度

洪明坚

重庆大学软件学院

February 19, 2019

- 1 CPU scheduling
 - Basic concepts
 - CPU or I/O burst
 - CPU scheduler
 - Scheduling criteria
- 2 Scheduling algorithms
 - First-come, first-served
 - Shortest Job First
 - Priority scheduling
 - Round-Robin scheduling
 - Multilevel queue
 - Multilevel feedback queue
 - Conclusion

Outline

1 CPU scheduling

- Basic concepts
- CPU or I/O burst
- CPU scheduler
- Scheduling criteria

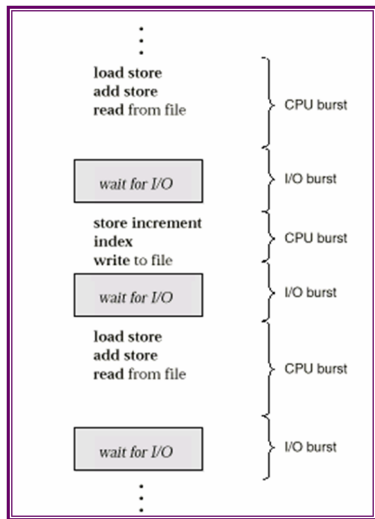
2 Scheduling algorithms

- First-come, first-served
- Shortest Job First
- Priority scheduling
- Round-Robin scheduling
- Multilevel queue
- Multilevel feedback queue
- Conclusion

- The multiprogramming is used by the operating system to overlap the operations of CPU and I/O devices in order to maximize CPU utilization.
 - When current running process can not proceed, the operating system must schedule one of the ready processes to run.
- Scheduling is a fundamental operating system function.
 - Almost all computer resources must be scheduled before use.
 - The CPU scheduling is central to operating system design.
- Operating systems supporting threads at the kernel level must schedule threads (NOT processes) for execution.

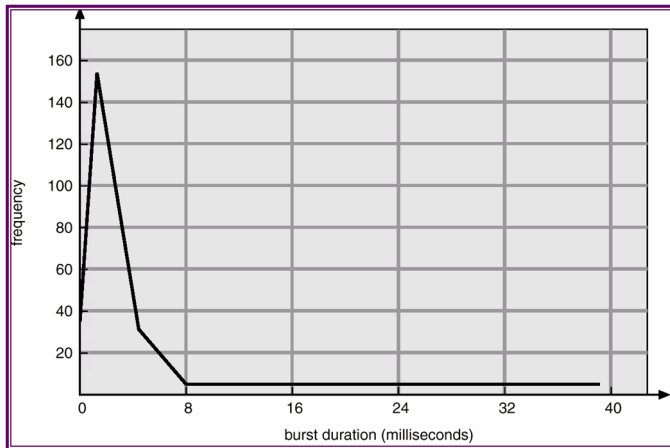
CPU-I/O burst cycle (1/2)

- The success of CPU scheduling depends on the following observed property of processes:
 - Process execution consists of a cycle of **CPU execution** and **I/O wait**.
 - Processes alternate between these two states.



CPU-I/O burst cycle (2/2)

- CPU burst distribution



- Selects from among the processes in memory that are ready to execute, and allocates the CPU to one of them.
 - Also known as **short-term scheduler**.
- CPU scheduling decisions may take place when a process:
 - ① Switches from running to waiting state.
 - ② Switches from running to ready state.
 - ③ Switches from waiting to ready.
 - ④ Terminates.
- Scheduling under 1 and 4 is **non-preemptive**. All other scheduling is **preemptive**.

Scheduling criteria

- CPU utilization – keep the CPU as busy as possible
- Throughput – # of processes that complete their execution per time unit
- Turnaround time – amount of time to execute a particular process
- Waiting time – amount of time a process has been waiting in the ready queue
- Response time – amount of time it takes from when a request was submitted until the first response is produced

Outline

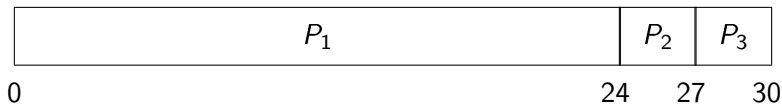
- 1 CPU scheduling
 - Basic concepts
 - CPU or I/O burst
 - CPU scheduler
 - Scheduling criteria
- 2 Scheduling algorithms
 - First-come, first-served
 - Shortest Job First
 - Priority scheduling
 - Round-Robin scheduling
 - Multilevel queue
 - Multilevel feedback queue
 - Conclusion

First-come, first-served (FCFS)

- Example

Process	Burst time
P_1	24
P_2	3
P_3	3

- Suppose that the processes arrive in the order: P_1 , P_2 , P_3 . The *Gantt Chart* for the schedule is:



- Waiting time for $P_1 = 0$; $P_2 = 24$; $P_3 = 27$
- Average waiting time: $(0 + 24 + 27)/3 = 17$

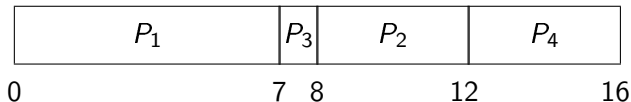
Shortest-Job-First (SJF)

- Associate with each process the length of its next CPU burst.
 - Use these lengths to schedule the process with the shortest time.
- Two schemes:
 - Non-preemptive - once CPU given to the process it cannot be preempted until completes its CPU burst.
 - Preemptive - if a new process arrives with CPU burst length less than remaining time of current executing process, preempt.
 - This scheme is known as the *Shortest-Remaining-Time-First (SRTF)*.
- SJF is optimal giving minimum average waiting time for a given set of processes.

Example: Non-preemptive SJF

Process	Arrival time	Burst time
P_1	0	7
P_2	2	4
P_3	4	1
P_4	5	4

- Non-preemptive SJF

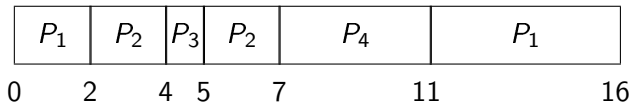


- Average waiting time = $(0 + 6 + 3 + 7)/4 = 4$.

Example: Preemptive SJF

Process	Arrival time	Burst time
P_1	0	7
P_2	2	4
P_3	4	1
P_4	5	4

- Preemptive SJF



- Average waiting time = $(9 + 1 + 0 + 2)/4 = 3$.

Priority scheduling

- A priority number (integer) is associated with each process
 - SJF is a priority scheduling where priority is the next CPU burst time.
- The CPU is allocated to the process with the highest priority.
 - It can be preemptive or non-preemptive
- Problems
 - ① *Starvation* - low priority processes may never execute.
 - ② *Priority inversion* - higher-priority process need to access a resource being held by another lower-priority process.
- Solution
 - *Aging* - increasing the priority of the process with time going on.

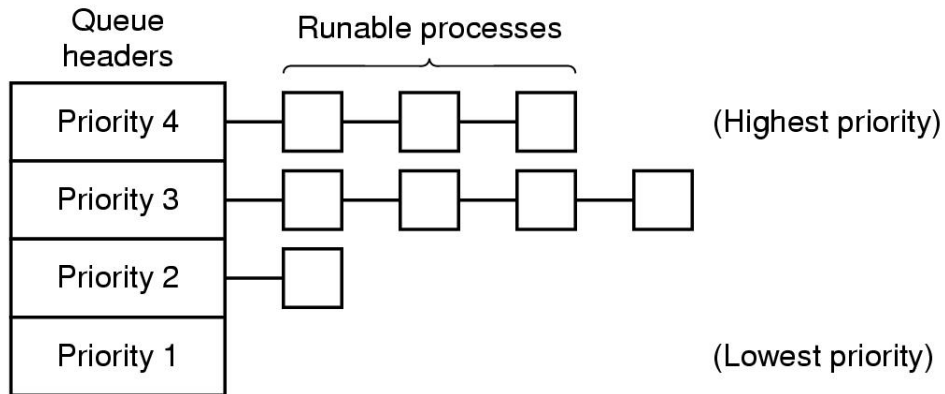
Round-Robin scheduling (RR)

- Each process gets a small unit of CPU time (time *quantum* or *slice*), usually 10-100 milliseconds.
 - After this time has elapsed, the process is preempted and added to the end of the ready queue.
- If there are n processes in the ready queue and the time quantum is q ,
 - then each process gets $1/n$ of the CPU time in chunks of at most q time units at once.
 - No process waits more than $(n-1)q$ time units.
- Performance
 - q large - FCFS
 - q small - q must be large with respect to context switch, otherwise overhead is too high.

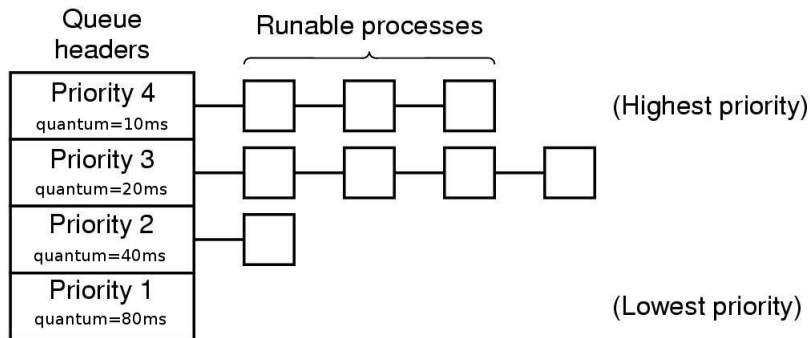
Multilevel queue (1/2)

- Ready queue is further partitioned into separate queues:
 - A priority number is associated with each queue.
 - Example: foreground (for interactive processes) and background (for batch processes) queues.
- Each queue has its own scheduling algorithm,
 - foreground uses RR; background uses FCFS, for example.

Multilevel queue (2/2)



Multilevel feedback queue scheduling



- A process can move between the various queues.
 - For example, whenever a process used up the quantum allocated to it, it sank into a lower priority queue.

Conclusion

- *Getting it right in practice is much harder than getting it right in principle.*
 - As a result, the scheduler rarely makes the best choice.
 - The solution: separation of scheduling policy and scheduling mechanism.
 - That is, the scheduling algorithm is parameterized in some way, but the parameters can be filled in by the user.

Questions

- Any questions?

